PROJECT REPORT

# E-commerce Customer Analytics

## SQL, Python and RFM Segmentation with K-Means

Dataset: UCI Online Retail (UK online gift retailer, 1 December 2010 to 9 December 2011)
Report date: 9 October 2026

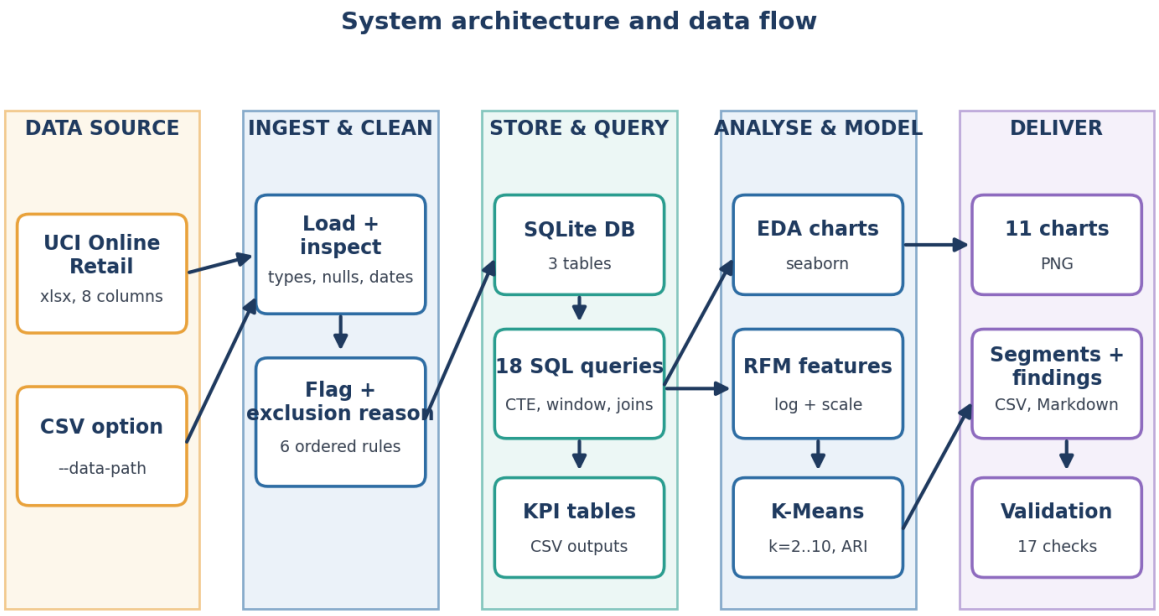



*Figure 1. System architecture: from the raw file to validated segments and recommendations.*

**Read this first: what the numbers in this report are**

The real UCI file (541,909 rows) was **not available** in the environment where the project was built, so the pipeline has **not been run on real data**. Facts about the dataset itself come from the project's dataset documentation, which marks them as verified against the UCI page. All results and charts in Sections 4 and 6 to 9 come from the project's own **synthetic test file** (25,707 rows, same eight columns). They show what the pipeline produces and that it works; they are **not findings about the real business** and must not be quoted as such. Charts carry a small "synthetic" tag.

# Executive summary

This project answers a practical retail question: **who are the valuable customers, how does revenue behave over time, and where should marketing effort go?** It does so with an end-to-end, reproducible pipeline built on the public UCI Online Retail dataset, a transaction log of a UK-based online gift retailer whose customers are largely wholesalers.

The pipeline loads the Excel file, **flags** every row against six ordered cleaning rules instead of deleting anything, stores all rows in a SQLite database, answers business questions with 18 SQL queries, then computes Recency, Frequency and Monetary (RFM) features per customer and groups customers with K-Means. It ends by profiling the clusters, drafting recommendations and running 17 automatic validation checks, including a reconciliation of SQL totals against pandas.

| Item | Detail |
|---|---|
| Source data | UCI Online Retail, CC BY 4.0; 541,909 rows, 8 columns, 22.6 MB (per UCI page) |
| Tech stack | Python 3.9+, pandas, NumPy, scikit-learn, matplotlib, seaborn, openpyxl; SQLite |
| Deliverables | 1 script (872 lines), 3 SQL files (18 queries), 11 charts, CSV tables, findings and validation reports, two guides |
| Method | Flag-first cleaning, SQL KPIs, log + standardised RFM, K-Means with k chosen by silhouette and minimum-size rule |
| Status | Tested end to end on synthetic data: 17 of 17 checks passed. Real data: not yet run |

## What the pipeline produced on the synthetic test file (illustrative)

- **Revenue concentration is a headline output.** On the test file the top 10% of customers hold 70.5% of identified revenue. Real data will differ; the point is that the project measures this.
- **Five segments were found** (k = 5, silhouette 0.42, stability ARI 0.997). Two small groups, "Champions" (7% of customers) and "At-risk high-value" (5%), carry 77% of revenue between them.
- **Half the customers are low-value and lapsed** (50% of customers, 6% of revenue), which suggests low-cost automation rather than paid campaigns.
- **Every recommendation is framed as an experiment** with a holdout group, because K-Means describes past behaviour and cannot prove that an action will work.

## What a reader should do next

Download the real file from the UCI page, place it in data/raw, run the script, and replace the illustrative figures in this report with the numbers in outputs/. Section 12 lists the exact steps.

# 1. Introduction and objectives

Online retailers collect large transaction logs but often lack a clear view of which customers matter most. Treating all customers alike wastes marketing spend on low-value accounts and risks neglecting the few that drive most of the revenue. This project builds the analytical foundation to separate them.

## Business questions

1. Which customers and products generate the most revenue and volume?
2. What are the monthly revenue, order and average-order-value trends?
3. How concentrated is revenue among customers, and how many customers buy only once?
4. Can customers be grouped by Recency, Frequency and Monetary value for targeted marketing?
5. Which segments deserve retention, loyalty or re-engagement actions, and what are the limits of the analysis?

## Scope guardrails

The project is deliberately cautious about what it claims. **Revenue is not profit** (the data has no costs). A long time since the last purchase is a **recency observation, not proof of churn**. K-Means is unsupervised, so it **describes** groups and does not predict. No uplift percentages are claimed for any campaign.

## Skills demonstrated

- **SQL (SQLite):** schema design with keys and CHECK constraints, GROUP BY / HAVING, INNER and LEFT JOIN, CASE WHEN, CTEs and window functions (RANK, ROW_NUMBER, NTILE, LAG).
- **Python:** pandas cleaning and feature engineering, matplotlib and seaborn charts, scikit-learn K-Means with evaluation and a stability test.
- **Business analytics:** KPI definitions, segmentation, recommendations with trade-offs, and honest statement of limitations.

# 2. Dataset

The UCI Online Retail dataset records all transactions between 1 December 2010 and 9 December 2011 for a UK-registered, non-store online retailer that mainly sells unique all-occasion gifts. Each row is one **line item**: one product on one invoice. An invoice (order) is made of one or more rows sharing an InvoiceNo.

| Property | Value |
|---|---|
| File / size | Online Retail.xlsx, 22.6 MB |
| Rows and columns | 541,909 rows; 8 columns (UCI header says 6 features because 2 are identifiers) |
| Period | 01 Dec 2010 to 09 Dec 2011; December 2011 is only partly covered |
| Currency | Pound sterling (GBP) |
| Licence | CC BY 4.0: credit the creators and indicate that the data was filtered |
| Source | archive.ics.uci.edu/dataset/352/online+retail; Chen, Sain and Guo (2012), doi:10.1057/dbm.2012.17 |

## Data dictionary

| Column | Meaning | Handling in the script |
|---|---|---|
| InvoiceNo | Invoice (order) number; a leading C marks a cancellation | Read as text so the C prefix survives |
| StockCode | Product code | Text; non-merchandise codes (postage, fees) are excluded |
| Description | Product name; may be missing or vary by code | Most frequent non-missing description per code |
| Quantity | Units on the line; negative for returns | Integer; Quantity <= 0 excluded from sales |
| InvoiceDate | Invoice timestamp | Datetime; basis for recency and monthly trends |

| Column | Meaning | Handling in the script |
|---|---|---|
| UnitPrice | GBP per unit; zeros occur | Decimal; UnitPrice <= 0 excluded |
| CustomerID | Pseudonymous customer number; blank on some rows | Nullable integer; used as an ID only |
| Country | Customer's country of residence | Text; not necessarily the delivery country |

**Privacy:** no names or contact details are documented. CustomerID is a pseudonymous identifier and must not be used to re-identify anyone.

## 3. Methodology and architecture

The project is a single Python script supported by three SQL files. Figure 1 (cover) shows the data flow. The twelve workflow stages are: download, load and inspect, flag and clean, build the SQLite database, run SQL KPIs, explore the data, engineer RFM features, log-transform and scale, run K-Means, profile and label clusters, write recommendations, and validate.

| Layer | Tool | Why it was chosen |
|---|---|---|
| Storage and queries | SQLite (sqlite3) | Free, serverless, single file; supports CTEs and window functions (3.25+) |
| Cleaning and features | pandas, NumPy | Transparent flagging, grouping and feature engineering |
| Modelling | scikit-learn K-Means | Fast, widely understood, works well on standardised numeric RFM features |
| Visualisation | matplotlib, seaborn | Eleven reproducible PNG charts written to outputs/charts |
| Reproducibility | Fixed seed, relative paths | Same result on any machine; --data-path, --workdir, --k, --seed options |

### Project folder layout

| Folder / file | Content |
|---|---|
| ecommerce*customer*analytics.py | Main script; run this |
| sql/ | 01*schema.sql*, *02*build*tables.sql*, *03*business_queries.sql |
| data/raw, data/processed | Downloaded file goes in raw; clean_sales.csv is written to processed |
| database/ | online_retail.db (created by the script) |
| outputs/ | charts/, tables/, rfm*customer*segments.csv, cluster*profile.csv*, *business*findings.md, validation_report.txt |
| docs/, tests/ | Two guides; make*synthetic*retail_data.py for a smoke test without the real file |

## 4. Data quality and cleaning

The cleaning strategy is **flag first, decide later**. Instead of deleting rows, each row receives boolean flags and exactly **one** exclusion reason: the first rule it matches (Figure 2). Because the rules are ordered, the rule counts add up to the raw row count, and nothing is lost: all rows stay in the database table raw_transactions.

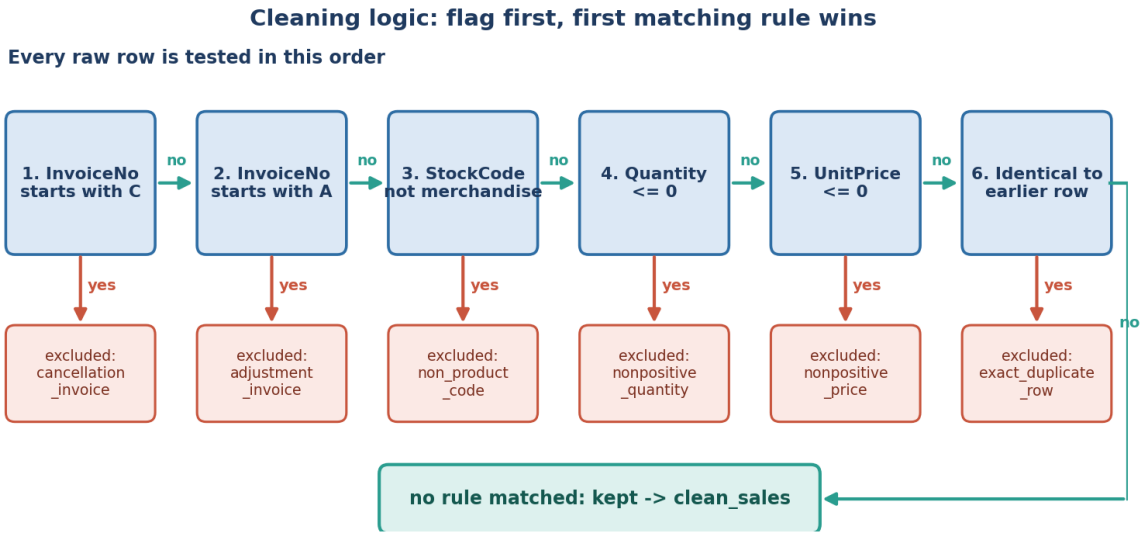



*Figure 2. Ordered exclusion rules. The first matching rule assigns the reason; unmatched rows are kept.*

| # | Rule | Justification |
|---|---|---|
| 1 | InvoiceNo starts with C | Cancellations cannot be linked to an original order, so they are not netted against customers |
| 2 | InvoiceNo starts with A | Accounting adjustments are not customer demand |
| 3 | StockCode is not 5 digits + 0 to 2 letters | Postage, fees and manual lines are not merchandise and distort rankings |
| 4 | Quantity <= 0 | Negative stock movement is not a sale |
| 5 | UnitPrice <= 0 | No revenue; likely free or adjustment lines |
| 6 | All 8 columns equal to an earlier row | Likely double entry; first occurrence kept (a documented assumption) |

**Deliberately NOT removed**

Rows with a missing CustomerID (real revenue, kept for revenue, product and country KPIs; left out only of customer-level RFM), large orders (likely genuine wholesale buying) and cancellations (kept in the database and analysed separately).

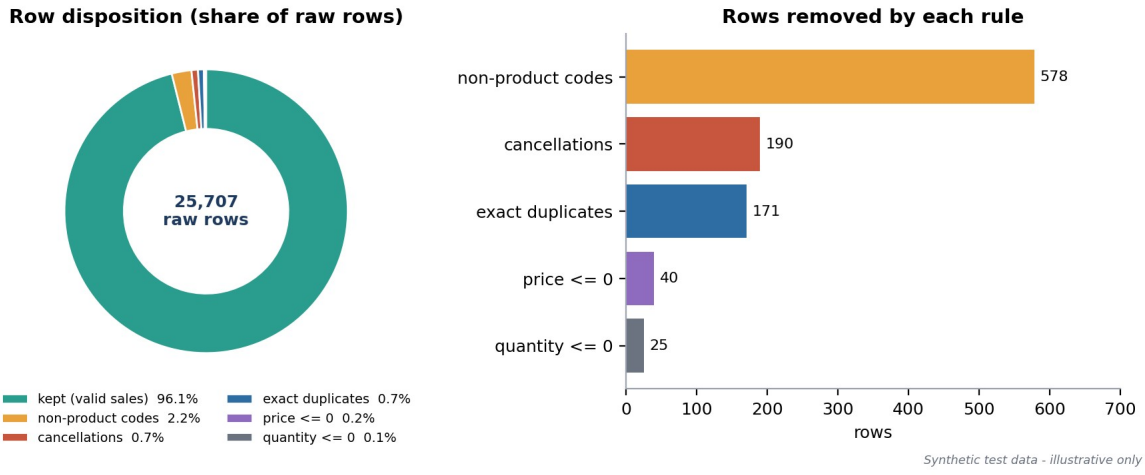



*Figure 3. Row disposition on the synthetic test file (25,707 raw rows; 96.1% kept).*

On the test file, non-product codes (postage, manual lines, bank charges) are the largest single exclusion at 578 rows, followed by cancellations (190) and exact duplicates (171). Missing CustomerID affects 1.2% of valid revenue. Real-data counts will appear in outputs/tables/exclusion_summary.csv.

## 5. SQL database and business queries

SQLite holds three tables (Figure 4). **raw_transactions** keeps every original row plus the flags; **products** maps each stock code to its most frequent description; **clean_sales** holds only valid sale lines and references products through a foreign key. CHECK constraints guard the flags and enforce positive quantity and price in clean_sales.

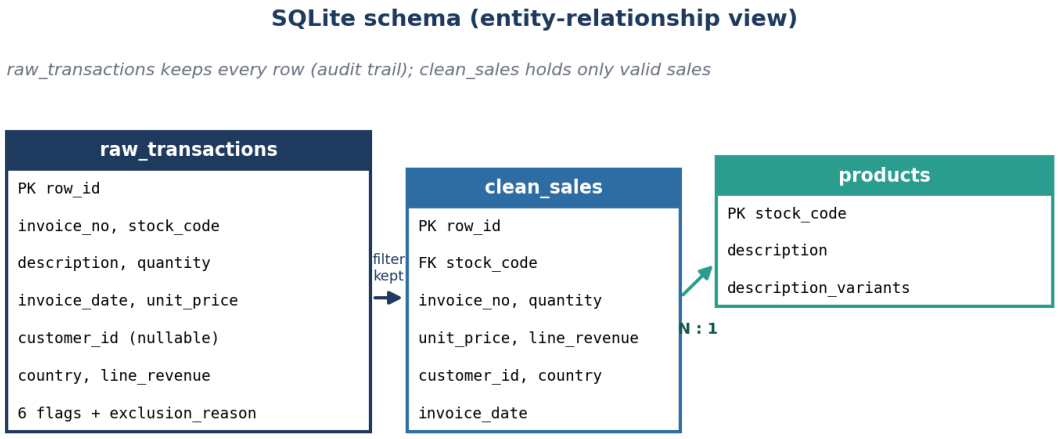



*Figure 4. SQLite schema: raw audit table, cleaned sales table and products lookup.*

The 18 named queries live in sql/03*business*queries.sql. The script splits the file at the name markers, runs each one and saves the result as a CSV. Python recomputes the key numbers independently, and the validation step compares the two.

| Query | Technique | Business question |
|---|---|---|
| q01, q02a/b | NULL checks, GROUP BY / HAVING | Where are missing values and duplicates? |
| q03 | CTE | Total revenue, orders, average order value |
| q04 | LAG window, strftime | Monthly KPIs with month-on-month growth |
| q05 | RANK, running SUM | Top customers and cumulative share |
| q06, q07 | INNER JOIN | Top products by revenue and by units |
| q08 | HAVING | Revenue by country (at least 5 orders) |
| q09, q10 | CASE WHEN, NTILE(10) | Frequency bands and revenue deciles |
| q11, q12 | LEFT JOIN, COALESCE | Cancellations vs sales; customers with no valid purchase |
| q13, q14 | julianday, NTILE + CASE | RFM table in SQL; rule-based quintile benchmark |
| q15, q16, q17 | Ratios, ROW_NUMBER | Repeat rate, top product per month, weekday revenue |

## 6. KPIs and exploratory analysis

The KPI definitions are fixed in advance so that every number can be explained. Revenue is Quantity x UnitPrice on valid sale lines; an order is a distinct InvoiceNo; both the mean and median order value are reported because a few very large wholesale orders inflate the mean.

| KPI (synthetic test file) | Value | How to read it |
|---|---|---|
| Total revenue | GBP 3,187,617 | Sales value, not profit |
| Orders / identified customers | 4,260 / 661 | One invoice = one order |
| Mean vs median order value | GBP 748 vs GBP 496 | Mean is 1.5x the median: a tail of large orders |
| Revenue with a CustomerID | 98.8% | Share of revenue usable for customer analysis |
| Repeat customers (2+ orders) | 71.7% | Loyalty proxy; late first-time buyers had little time to return |

| KPI (synthetic test file) | Value | How to read it |
|---|---|---|
| Cancelled value / valid sales | 0.14% | Revenue-leakage proxy; not linkable to original orders |
| Top 10% of customers | 70.5% of revenue | Dependence on a few accounts |

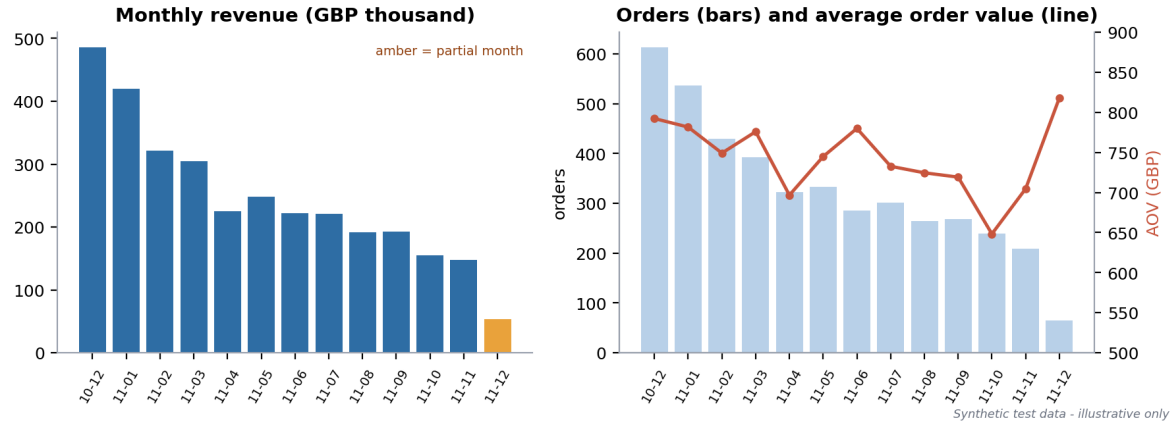



*Figure 5. Monthly revenue, orders and average order value. The partial final month is marked in amber.*

Partial months are flagged on charts and excluded when picking the best month. One year of data cannot separate seasonality from growth, so a falling line in a synthetic file must not be read as a trend.

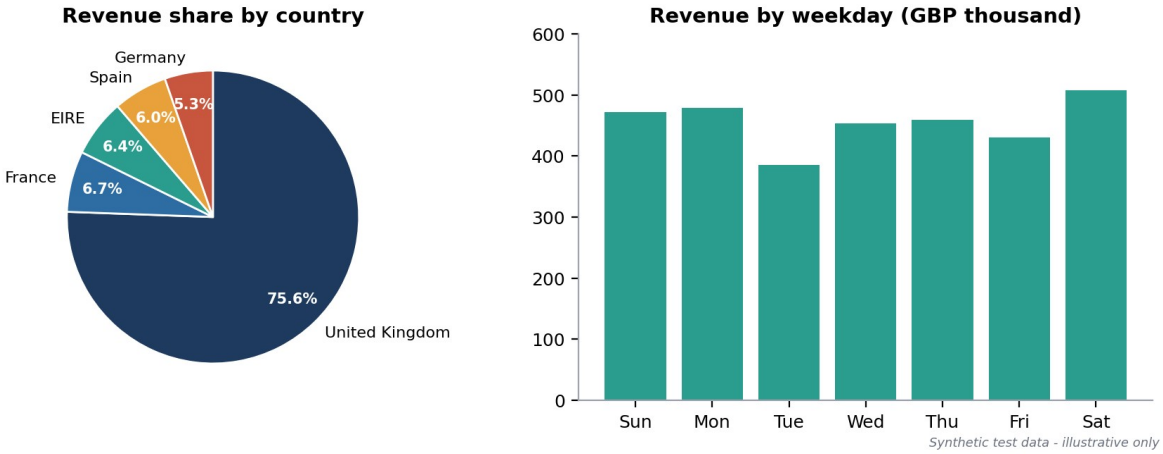



*Figure 6. Revenue share by country and revenue by weekday.*

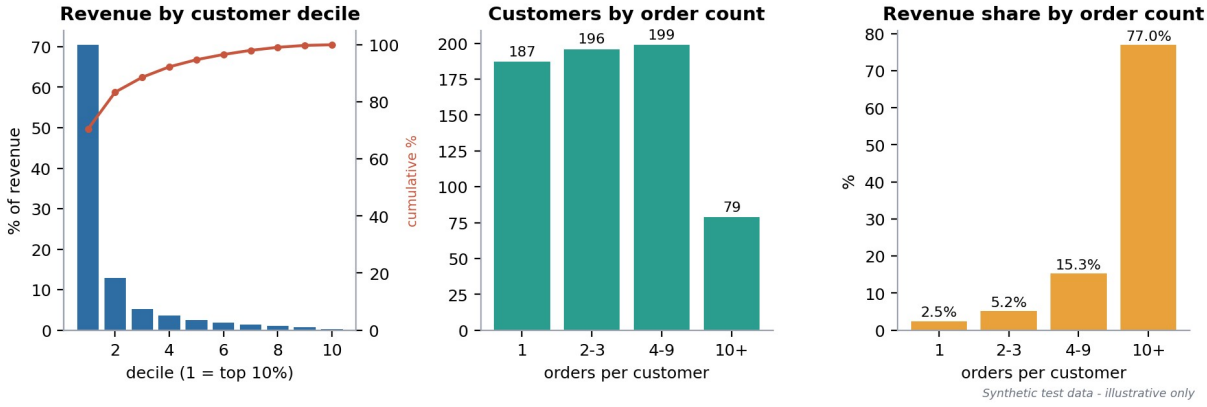



*Figure 7. Revenue concentration by customer decile, and customer and revenue split by number of orders.*

Figure 7 shows why the project reports concentration: customers with ten or more orders are 12% of identified customers but 77% of revenue on the test file. Such a skew is also why the later modelling step uses a log transform.

## 7. RFM features and K-Means segmentation

Because there is no label such as a churn flag, the right tool is **unsupervised** learning. A classifier would need a defined target (for example, bought again within 90 days), a prediction horizon and time-based validation, none of which the data provides.

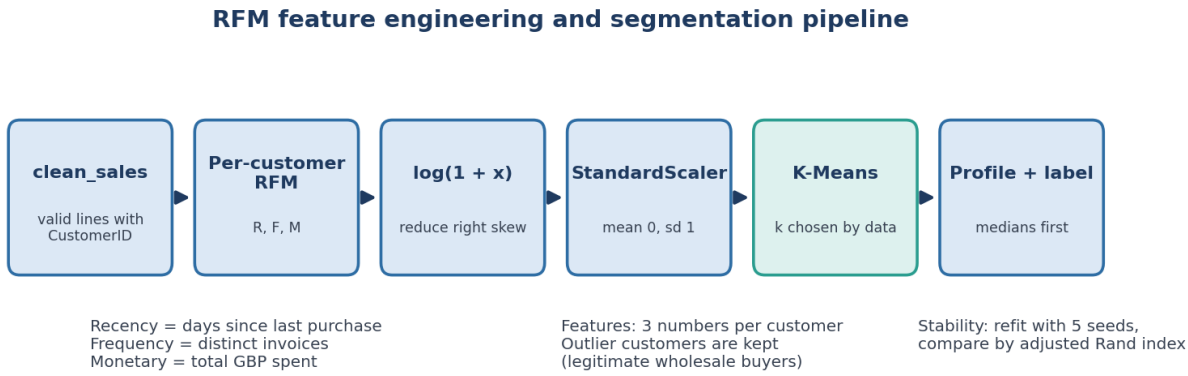



*Figure 8. From valid sales to labelled segments.*

| Variable | Exact calculation |
|---|---|
| Snapshot date | Midnight after the last valid sale; fixed by the data so results are reproducible |
| Recency (days) | Whole days between the customer's last valid purchase and the snapshot (lower is more recent) |
| Frequency | Number of distinct valid InvoiceNo values for the customer |
| Monetary (GBP) | Sum of valid line revenue over the whole window; cancellations are not netted off |
| Model features | log(1 + R), log(1 + F), log(1 + M), then standardised to mean 0 and SD 1 |

**Choosing k.** The script fits k = 2 to 10 and records inertia, silhouette, Davies-Bouldin and the smallest cluster's share. The rule is: take the highest silhouette among k >= 3 whose smallest cluster holds at least 2% of customers (k = 2 is usually too coarse for targeting). The elbow chart is a visual sanity check, and --k overrides the choice.

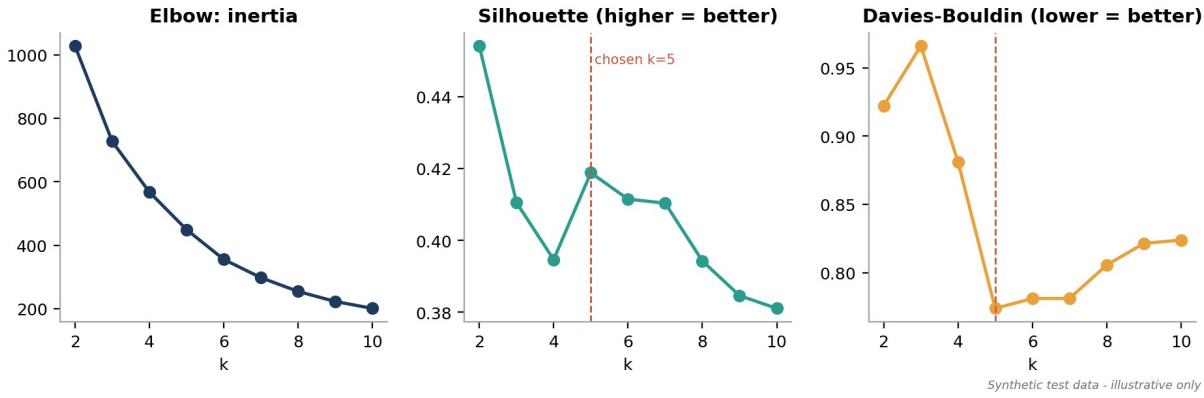



*Figure 9. Cluster-count selection on the synthetic file. k = 5 is chosen.*

| Metric | Value (k = 5) | Meaning |
|---|---|---|
| Silhouette | 0.419 | Moderate; RFM data is a continuum, so moderate values are normal |
| Davies-Bouldin | 0.774 | Lower is better; k = 5 is the minimum of the range |
| Stability (ARI, 5 reseeds) | mean 0.997, min 0.985 | Near 1: the same grouping is found with different seeds |

## 8. Segment results (illustrative)

Clusters are renumbered so that cluster 0 has the highest median Monetary value. Labels are applied **only after profiling**: a cluster counts as recent if its median Recency is below the overall median, and as frequent or high-spend if its median reaches the overall 75th percentile. The names are suggestions to be reviewed against the profile.

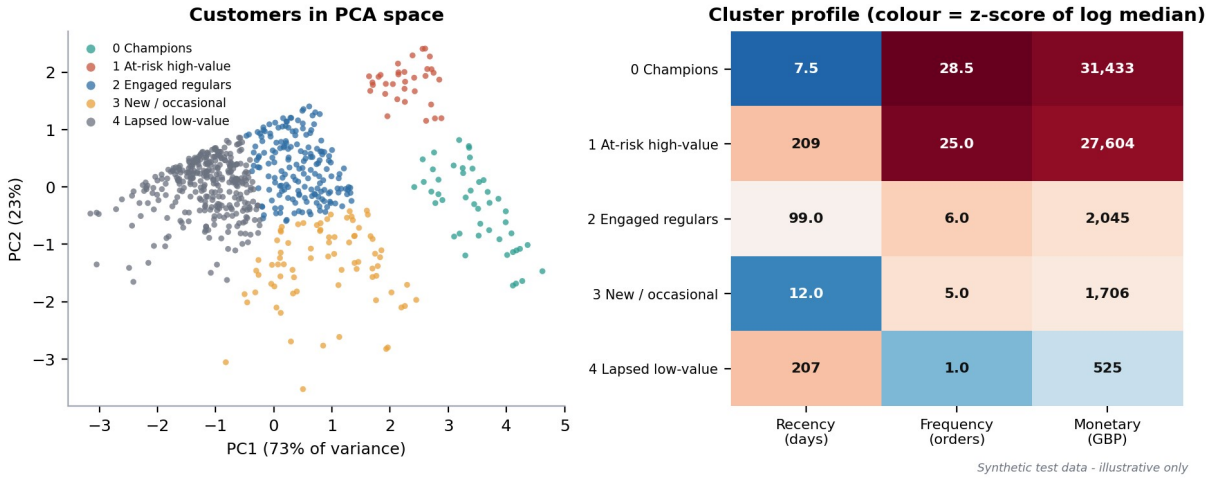



*Figure 10. Left: customers projected onto two principal components. Right: cluster profile (text = actual medians; colour = standardised level, red is high).*

| Cluster / label | Customers | Median R / F / M | % customers | % revenue |
|---|---|---|---|---|
| 0 Champions | 46 | 7.5 d / 28.5 / GBP 31,433 | 7.0% | 46.0% |
| 1 At-risk high-value | 33 | 209 d / 25 / GBP 27,604 | 5.0% | 31.0% |
| 2 Engaged regulars (lower spend) | 176 | 99 d / 6 / GBP 2,045 | 26.6% | 12.5% |
| 3 New or occasional recent buyers | 77 | 12 d / 5 / GBP 1,706 | 11.7% | 4.7% |
| 4 Lapsed low-value | 329 | 207 d / 1 / GBP 525 | 49.8% | 5.7% |

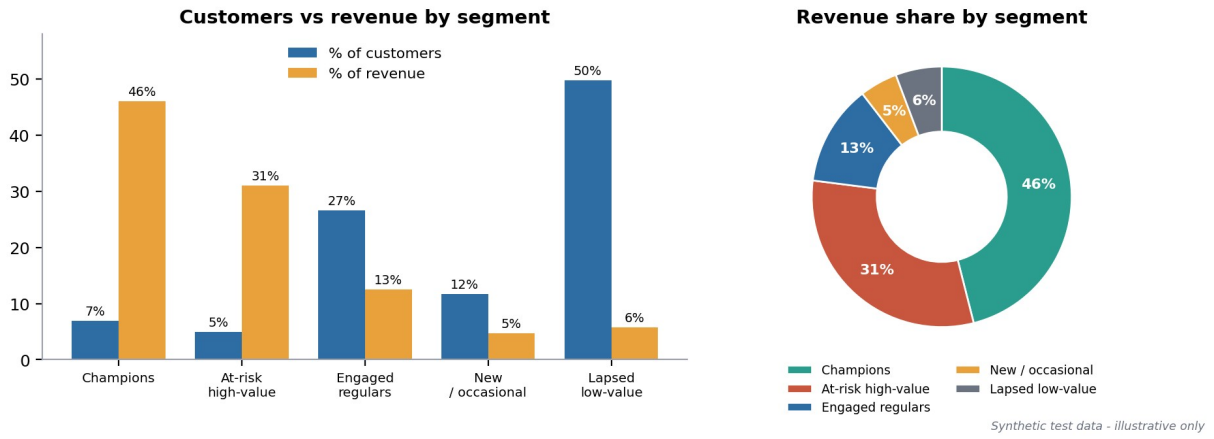



*Figure 11. Share of customers against share of revenue for each segment.*

The gap between the blue and amber bars is the key message: the two high-value groups are 12% of customers and 77% of revenue, while the lapsed low-value group is half of all customers and under 6% of revenue. The project also computes a transparent rule-based quintile benchmark (q14) for managers to compare with the K-Means result.

## 9. Validation

The script ends with 17 automatic checks and exits with code 0 only if all pass. The most important ones reconcile SQL against pandas, which guards against a silent logic error in either.

| Check group | What is verified | Result on test run |
|---|---|---|
| Row accounting | Exclusion reasons cover every raw row (25,707) | PASS |
| SQL vs pandas totals | Revenue (GBP 3,187,617.19 on both sides) and order counts agree | PASS (2 checks) |
| SQL vs pandas RFM | Same 661 customers; Recency, Frequency and Monetary (+/- 0.01) match | PASS (4 checks) |
| Feature sanity | No NaN or inf; Recency >= 0, Frequency >= 1, Monetary > 0 | PASS (2 checks) |
| Model sanity | Every customer has a cluster; no empty cluster; silhouette in [-1, 1]; sizes sum to customers | PASS (4 checks) |
| Outputs | Segments, profile and findings files exist; at least 10 charts saved | PASS (4 checks) |

**What was and was not tested**

Tested: .xlsx and .csv input, the --k override, the missing-file message, all 18 SQL queries, 11 charts and 17 checks on synthetic data (pandas 3.0.2, scikit-learn 1.8.0). **Not tested:** the real UCI file, real-data runtime and quirks, Windows-specific behaviour, older pandas 2.x or scikit-learn versions, and CSVs with other date formats.

## 10. Recommendations

Recommendations are hypotheses to test, not guaranteed outcomes. The wording below follows the suggested playbook in the script; the segment names and sizes are from the synthetic run and will change with real data.

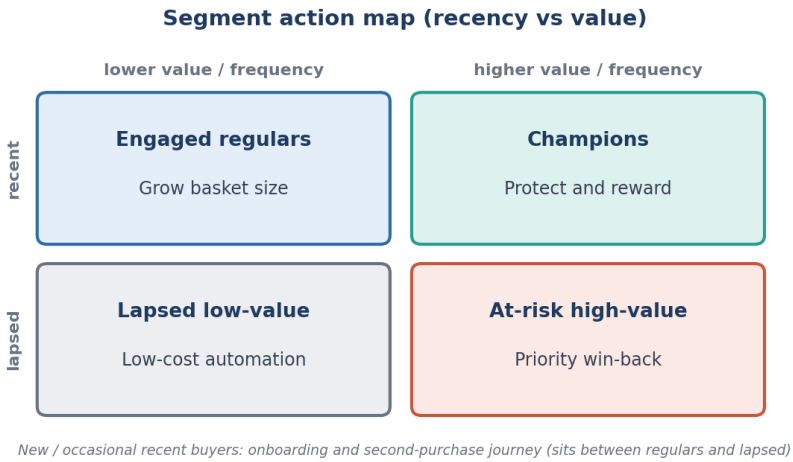



*Figure 12. Segment action map. The positions are conceptual, based on recency and value.*

| Segment | Suggested action | How to test it |
|---|---|---|
| Champions | Early access, loyalty tiers and service perks; avoid blanket discounts | Non-discount reward vs holdout; metric: repeat rate and revenue per customer |
| At-risk high-value | Personal outreach or account manager, ask for feedback, capped time-limited incentive | Randomised holdout; track reactivation within a fixed window and margin |
| Engaged regulars | Bundles, cross-sell, free-delivery or volume thresholds | A/B test of the threshold; metric: average order value |
| New / occasional | Welcome series and second-purchase journey based on the first basket | Email journey vs control; metric: second-order rate |
| Lapsed low-value | Low-cost automated reactivation only; suppress from paid campaigns | Compare automated vs no contact; cap contact frequency |

Further actions: assign relationship owners to top-decile accounts and set early-warning triggers when the gap between their orders grows; capture cancellation reasons and link them to the original invoice; and add margin, channel and campaign data before optimising spend.

# 11. Limitations and next steps

## Limitations

- One UK gift retailer and about 12 months of data (2010 to 2011): results do not generalise, and one year cannot separate seasonality from growth.
- No cost, margin, discount, channel, category or delivery data: profitability and campaign effectiveness cannot be measured.
- Many customers are wholesalers, so behaviour differs from typical consumers.
- Rows without CustomerID are excluded from customer analysis; if those buyers differ, customer-level results are biased.
- RFM data is a continuum. K-Means gives useful partitions rather than discovered natural groups, and Recency, Frequency and Monetary depend on the snapshot date.

## Conclusions the data does not support

That any customer has churned; profit, margin or ROI of any action; that a campaign will raise revenue by a given percentage; causal statements such as discounts causing higher spend; and any product-category insight.

## Next steps

1. Run the pipeline on the real file and replace the illustrative numbers in this report.
2. Add margin data so segments can be ranked by profit.
3. Test whether segments are stable over time with more periods.
4. Separate wholesale and retail behaviour.
5. Build a proper repeat-purchase model with a defined target and a time-based train and test split.
6. Run randomised experiments on the proposed actions.

# 12. How to reproduce, and references

| Step | Command or action (Windows PowerShell) |
|---|---|
| 1 | python -m venv .venv  then  .venv\Scripts\activate |
| 2 | pip install -r requirements.txt |
| 3 | Download Online Retail.xlsx from the UCI page; place it in data\raw\ (exact name) or use --data-path |
| 4 | python ecommerce*customer*analytics.py  (options: --k 4, --seed 7, --workdir) |
| 5 | Open outputs\validation_report.txt; all checks should pass. Use outputs\ for your real numbers |
| Smoke test | python tests\make*synthetic*retail*data.py, then run the script with --data-path "tests\synthetic*data\Online Retail.xlsx" --workdir test_run |

## References

Chen, D., Sain, S. L. and Guo, K. (2012). Data mining for the online retail industry: a case study of RFM model-based customer segmentation using data mining. Journal of Database Marketing & Customer Strategy Management, 19(3), 197-208. doi:10.1057/dbm.2012.17

Chen, D. Online Retail dataset. UCI Machine Learning Repository, CC BY 4.0. https://archive.ics.uci.edu/dataset/352/online+retail. The data was filtered and cleaned for this project.

Project files: ecommerce*customer*analytics.py, sql/*.sql, docs/step*by*step*project*guide.docx, docs/dataset*documentation.docx*, *docs/build*validation_summary.txt.